ELEVATE LABS DEVOPS INTERNSHIP • MINISTRY OF MSME, GOVT. OF INDIA • PROJECT #3

Complete Observability System

Executive Engineering Report — Unifying Prometheus Metrics, Loki Centralized Logs, and Jaeger Distributed Tracing with OpenTelemetry Instrumentation & Declarative Grafana Dashboards

AUTHOR & MAINTAINER
Preetham Pereira
cloudwithpreetham • MIT 2026

ORCHESTRATION
Docker Compose
6 Containerized Services

APPLICATION RUNTIME
Python 3.11 / Flask 3.0
OTel SDK 1.25 • Port 5000

VERIFICATION STATUS
Deployed & Verified
26 Git Commits • Live Telemetry

1 Executive Summary & Problem Statement

Traditional infrastructure monitoring answers only whether a host or process is reachable (*"Is it up?"*), leaving engineering teams blind when diagnosing tail latency, intermittent errors, and cascading bottlenecks in distributed microservices. Modern Site Reliability Engineering (SRE) replaces siloed health checks with **observability** — the ability to infer internal execution state purely from correlated external telemetry outputs.

This project delivers a self-contained, single-command Observability Minimum Viable Product (`docker compose up -d --build`) built around a custom instrumented Python Flask microservice (`flask-observability-app`). By simultaneously exposing time-series metrics, shipping trace-annotated container logs, and exporting OpenTelemetry spans via OTLP gRPC, the platform unifies all three pillars inside pre-provisioned Grafana dashboards with zero manual UI configuration.

1. Metrics Pillar — Prometheus v2.52

Pull model (5s interval): Scrapes `/metrics` exposed by `prometheus-flask-exporter` , capturing HTTP request counters, status codes, and latency histograms into TSDB.

2. Logging Pillar — Loki & Promtail v3.0

Push model (Docker SD): Promtail tails container stdout via `docker.sock` , extracts `container` labels, and ships trace-correlated logs to Loki's v13 TSDB store.

3. Tracing Pillar — Jaeger v1.57 + OTel

Push model (OTLP gRPC): OpenTelemetry SDK auto-instruments Flask requests and exports custom spans (`compute_job` , `slow_database_query`) to Jaeger port `4317` .

2 Containerized Technology Stack & Service Topology

Service / Container	Image / Version	Ports & Network	Mounts / Configuration	Architectural Role
flask-app `flask-observability-app`	Built from `./app` Python 3.11-slim	`5000:5000` Depends: jaeger	`OTEL_PYTHON_LOG_CORRELATION=true`	Simulated microservice exposing workloads, Prometheus metrics, OTLP traces, and trace-injected logs.
prometheus `prometheus`	`prom/prometheus:v2.52.0`	`9090:9090`	`prometheus/prometheus.yml` TSDB: `/prometheus`	Time-series database polling `flask-app:5000/metrics` and self-monitoring every 5 seconds.
loki `loki`	`grafana/loki:3.0.0`	`3100:3100`	`loki/loki-config.yml` Schema v13 TSDB	Centralized log aggregator indexing container label metadata with filesystem chunk & 24h index storage.
promtail `promtail`	`grafana/promtail:3.0.0`	Internal `9080` Depends: loki	`promtail-config.yml` `docker.sock:ro`	Discovers running Docker containers every 5s and pushes stdout log streams to Loki Push API.
jaeger `jaeger`	`jaegertracing/all-in-one:1.57`	`16686:16686` (UI) `4317:4317` (gRPC)	`COLLECTOR_OTLP_ENABLED=true`	OTLP gRPC collector, trace storage backend, and waterfall call-graph visualizer.
grafana `grafana`	`grafana/grafana:10.4.2`	`3000:3000` Depends: 3 backends	`grafana/provisioning/` Admin / Anonymous Admin	Unified visualization UI auto-provisioning Prometheus, Loki, and Jaeger datasources plus 4-panel dashboard.

Repository Engineering Progression: Git commit history (26 commits) reflects iterative DevOps refinement — progressing from container build optimization (`.dockerignore`) and pipeline configs to targeted bug fixes including dynamic `LogRecordFactory` trace-ID injection (resolving Werkzeug formatting exceptions) and deterministic Grafana datasource UID normalization.

3 System Architecture & Telemetry Pipelines

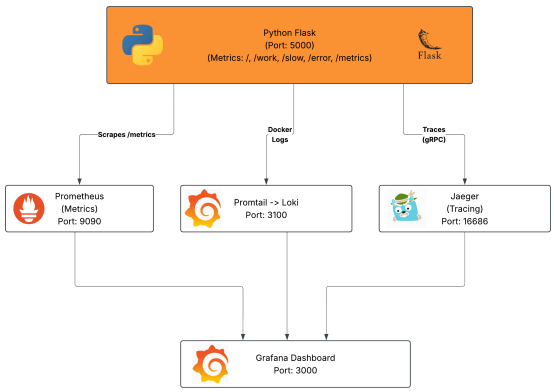


Figure 1: End-to-End Observability Architecture (`docs/architecture/observability-architecture.png`) showing the instrumented Flask service feeding Prometheus, Promtail/Loki, and Jaeger into Grafana.

Metrics Pull Pipeline

`prometheus.yml` defines global `scrape_interval: 5s` and `evaluation_interval: 5s` across two static jobs: `flask-app` (`flask-app:5000/metrics`) and `prometheus` (`localhost:9090`). Metrics include `app_info` , `flask_http_request_total` , and duration buckets.

Decoupled Log Shipping

Flask writes logs unbuffered (`PYTHONUNBUFFERED=1`) to stdout. Promtail mounts `/var/run/docker.sock` and `/var/lib/docker/containers` read-only, relabeling `__meta_docker_container_name` to `container` and shipping to `loki:3100/loki/api/v1/push` .

OTLP Distributed Tracing

`TracerProvider` tags spans with `service.name="flask-observability-app"` and streams via `BatchSpanProcessor` + `OTLPSpanExporter` over gRPC (`jaeger:4317`). Both auto-instrumented HTTP spans and manual workload spans are captured.

4 Flask Application Instrumentation & Endpoint Specifications

The target microservice (`app/app.py` , 80 lines) uses `Flask==3.0.3` , `prometheus-flask-exporter==0.23.1` , and OpenTelemetry SDK `1.25.0` (`0.46b0` instrumentation packages). Five routes model normal traffic, compute work, slow database queries, and server faults:

Endpoint	HTTP Status	Simulated Workload / Latency	OpenTelemetry Span	Log & Metric Telemetry Emitted
`GET /`	200 OK	Service index & endpoint list	Auto HTTP root span	`INFO` log + request counter
`GET /work`	200 OK	Batch compute (`50–250 ms`)	Child span `compute_job`	`INFO` latency log + duration histogram
`GET /slow`	200 OK	Slow DB query (`800–1500 ms`)	Child span `slow_database_query`	`WARNING` alert log + latency tail spike
`GET /error`	500 ERR	Simulated internal server fault	Auto HTTP span (`status=500`)	`ERROR` log + HTTP 500 metric counter
`GET /metrics`	200 OK	Prometheus exposition endpoint	Auto HTTP span (scraped /5s)	Counter, histogram & `app_info` gauges

APP/APP.PY — TRACE-TO-LOG CONTEXT INJECTION & OTLP SETUP

```
# Safely inject active Trace ID & Span ID into every Python LogRecord
old_factory = logging.getLogRecordFactory()
def record_factory(*args, **kwargs):
    record = old_factory(*args, **kwargs)
    span = trace.get_current_span()
    ctx = span.get_span_context() if span else None
    if ctx and ctx.is_valid:
        record.otelTraceID = format(ctx.trace_id, "032x")
        record.otelSpanID  = format(ctx.span_id,  "016x")
    else:
        record.otelTraceID, record.otelSpanID = "0"*32, "0"*16
    return record
logging.setLogRecordFactory(record_factory)

# OTLP gRPC Exporter & Flask/Prometheus Instrumentation
res = Resource(attributes={"service.name": "flask-observability-app"})
provider = TracerProvider(resource=res)
exporter = OTLPSpanExporter(endpoint="jaeger:4317", insecure=True)
provider.add_span_processor(BatchSpanProcessor(exporter))
trace.set_tracer_provider(provider)
LoggingInstrumentor().instrument()
FlaskInstrumentor().instrument_app(app)
metrics = PrometheusMetrics(app)
```

Why the Custom `LogRecordFactory` Matters

Format strings expecting `%(otelTraceID)s` raise `KeyError` when background loggers (e.g. Werkzeug's HTTP logger) emit events outside an active span.

Wrapping `getLogRecordFactory()` ensures every record includes a 32-hex `trace_id` and 16-hex `span_id` (or zero fallback) for reliable Promtail/Loki ingestion and cross-pillar pivoting.

Synthetic Load (`traffic-generator.sh`)

Loops continuously across `/` , `/work` , `/slow` , and `/error` at `0.3s` intervals, generating realistic throughput curves, tail-latency spans, warning logs, and HTTP 500 errors.

5 Declarative Grafana Provisioning & Telemetry Queries

Grafana (`v10.4.2`) boots completely pre-configured via mounted YAML and JSON provisioning artifacts. `datasources.yml` assigns deterministic UIDs (`prometheus` , `loki` , `jaeger`) using proxy mode over internal Docker DNS, while `dashboards.yml` loads `dashboard.json` (UID `eg2gkti31bls0b` , 10s refresh scan) presenting four correlated telemetry panels:

Panel 1: HTTP Request Throughput (req/sec) — PromQL • uid: prometheus

```
sum by (endpoint) (rate(flask_http_request_total[1m]))
```

Panel 2: Average Request Latency (seconds) — PromQL • uid: prometheus

```
rate(flask_http_request_duration_seconds_sum[1m]) / rate(flask_http_request_duration_seconds_count[1m])
```

Panel 3: Application Centralized Logs — LogQL • uid: loki

```
{container="flask-observability-app"}
```

Panel 4: Distributed Trace Waterfall — Jaeger • uid: jaeger

```
service: "flask-observability-app" | limit: 20
```

6 Live Verification & Runtime Evidence

The stack was deployed via Docker Compose and exercised using the synthetic traffic generator. All six service containers reached `Up` state and every telemetry pipeline was validated across Prometheus, Jaeger, Loki, and Grafana:

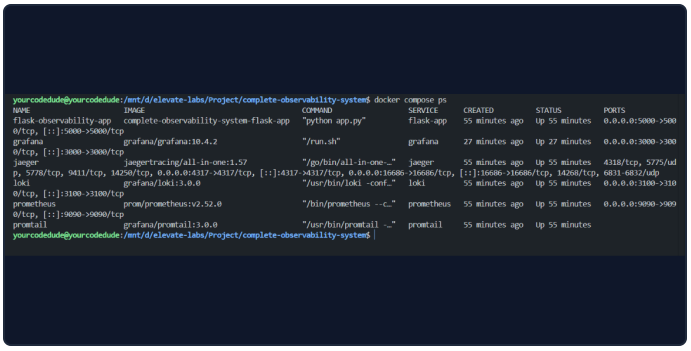


Figure 2 — Container Orchestration (`docker compose ps`): All 6 services (`flask-observability-app` , `grafana` , `jaeger` , `loki` , `prometheus` , `promtail`) active with host ports bound (`5000, 3000, 16686, 4317, 3100, 9090`).

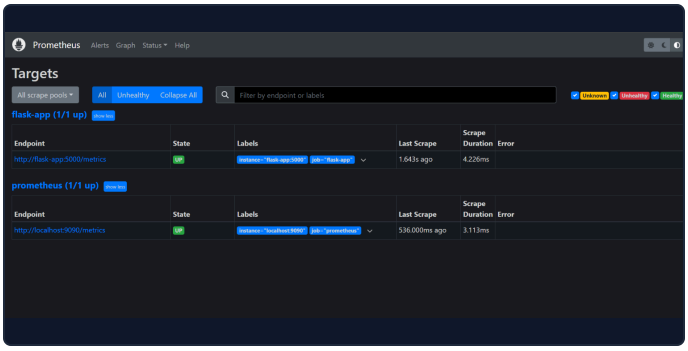


Figure 3 — Prometheus Target Health: Both `flask-app` (`http://flask-app:5000/metrics` , scrape duration `4.226 ms`) and `prometheus` (`3.113 ms`) report healthy `UP` status at 5s polling intervals.

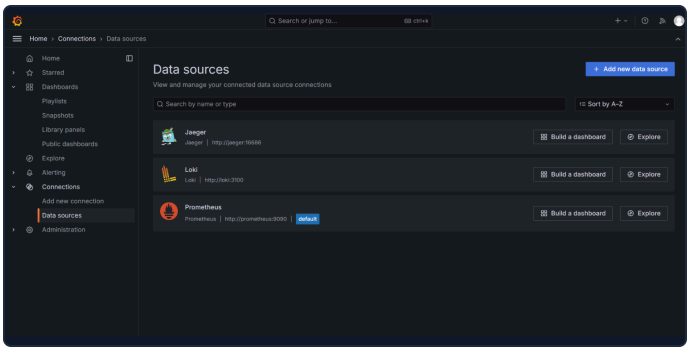


Figure 4 — Grafana Provisioned Datasources: Zero-touch YAML provisioning registers **Jaeger** (`jaeger:16686`), **Loki** (`loki:3100`), and **Prometheus** (`prometheus:9090` , marked `default`).

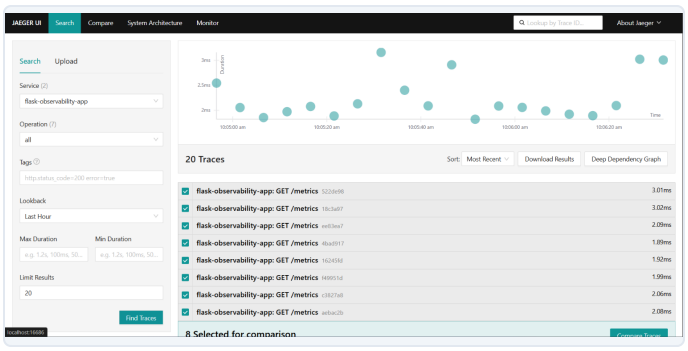


Figure 5 — Jaeger Trace Ingestion: Jaeger UI Search shows `flask-observability-app` active with `7` operations and `20` recent traces plotted by duration (~ `1.89–3.02 ms` for `GET /metrics` scrapes).

6 Verification Evidence (Continued): Logs & Unified Correlation

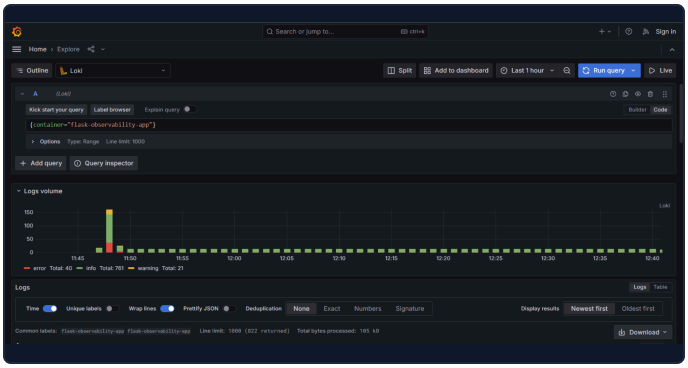


Figure 6 — Grafana Explore (Loki Log Stream): Query `{container="flask-observability-app"}` displays live log volume distribution across severities: **info: 761**, **error: 40**, and **warning: 21** (`822` lines, `105 kB` processed).

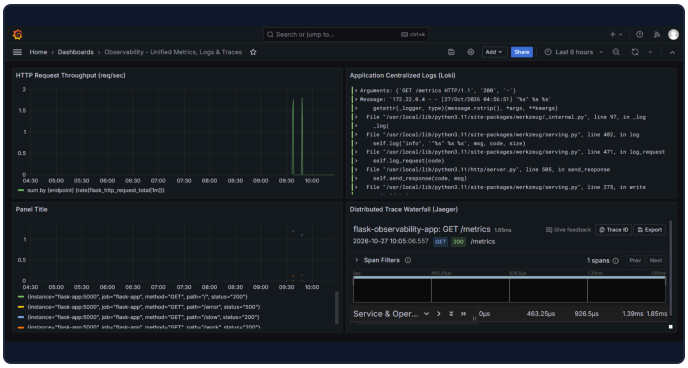


Figure 7 — Unified Grafana Dashboard: Four-quadrant view correlating Prometheus throughput & endpoint latency (`/` , `/work` , `/slow` , `/error`), Loki logs, and Jaeger waterfall (`1.85 ms`). *Note:* Captured prior to commits `9b85787` / `3c3b8b6` that refined log trace injection & panel titles.

7 Key Results & Production Hardening Roadmap

✓ Key Outcomes & Triage Workflow

- **Zero-Touch Provisioning:** Single `docker compose up -d --build` launches all 6 services with pre-linked datasources and dashboard UIDs.
- **Root-Cause Triangulation:** PromQL highlights `/slow` latency (`0.8–1.5s`) or `/error` 500 spikes; engineers inspect the `slow_database_query` span in Jaeger and match its 32-hex `trace_id` directly in Loki.
- **Low-Overhead Log Indexing:** Loki indexes only container metadata labels rather than full-text payloads, minimizing CPU and memory footprint.
- **Automated Validation:** Traffic harness confirmed simultaneous ingestion across Prometheus, Loki, and Jaeger.

⚠ Codebase Audit & Production Hardening

- **Persistent Storage Volumes:** Add named Docker volumes for `/prometheus` , `/loki` , and `/var/lib/grafana` so telemetry survives container recreation.
- **Security Posture:** Disable `GF_AUTH_ANONYMOUS_ENABLED=true` , rotate `admin/admin` , enable TLS on Jaeger OTLP gRPC (`4317`), and replace Promtail's host `docker.sock` mount with a scoped log driver.
- **WSGI Server & Trace Noise Filtering:** Run Flask behind Gunicorn instead of Werkzeug's dev server, and pass `excluded_urls="/metrics"` to `FlaskInstrumentor` so 5s Prometheus scrapes do not pollute Jaeger traces.
- **Healthchecks & Alerting:** Define Compose `healthcheck` probes and add Prometheus/Grafana alerts for 5xx error rates and p95 latency SLOs.

8 Conclusion & Repository Deliverables Index

The **Complete Observability System** satisfies all Elevate Labs DevOps Internship Project #3 objectives by bridging metrics, logs, and distributed traces into an integrated, reproducible architecture. By eliminating telemetry silos and embedding OpenTelemetry trace context directly inside application logs, the system provides a practical, extensible blueprint for reducing Mean Time to Detection (MTTD) and Mean Time to Resolution (MTTR) in cloud-native environments.

`docker-compose.yml` 6-service stack orchestration, ports & configs	**`app/app.py`** & **`Dockerfile`** Instrumented Flask microservice & OTel hooks	**`prometheus/prometheus.yml`** 5s scrape jobs for Flask & self-metrics
`loki/` & **`promtail/`** TSDB v13 store & Docker SD log shipping	**`grafana/provisioning/`** Automated datasources & 4-panel dashboard JSON	**`scripts/traffic-generator.sh`** Synthetic load testing & fault injection